



OS Development

7. Filesystem

Prof. Dr. Michael Schöttner
Dr. Fabian Ruhland

- Physical vs. Virtual Filesystems

- Filesystem API

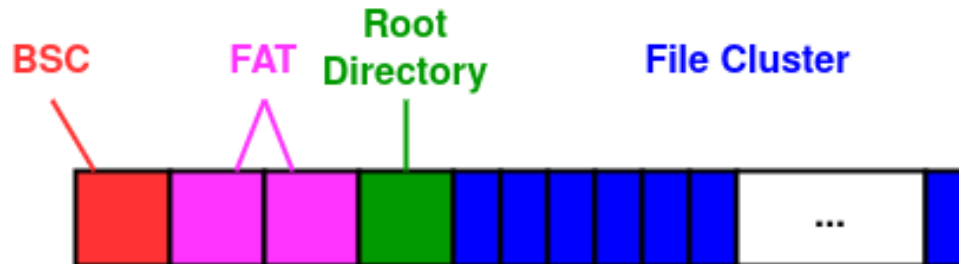
- HeineOS Filesystem

- Summary

- Persistent data is stored on disk drives (HDDs, SSDs, USB-Sticks, etc.)
- Problems:
 - Disk drives are usually organized in sectors (e.g. 512 bytes) and not byte-addressable (also called *block storage*)
 - Allocation/Deallocation of sectors for different files
 - File can be fragmented across multiple sectors
- Solution: Filesystems take care organizing files on disks:
 - Allocate sectors when writing to a file (and deallocating them when files are deleted)
 - OS provides file API allowing byte-by-byte read/write operations → Must be translated into sector operations
 - Usually a block cache is used for performance reasons

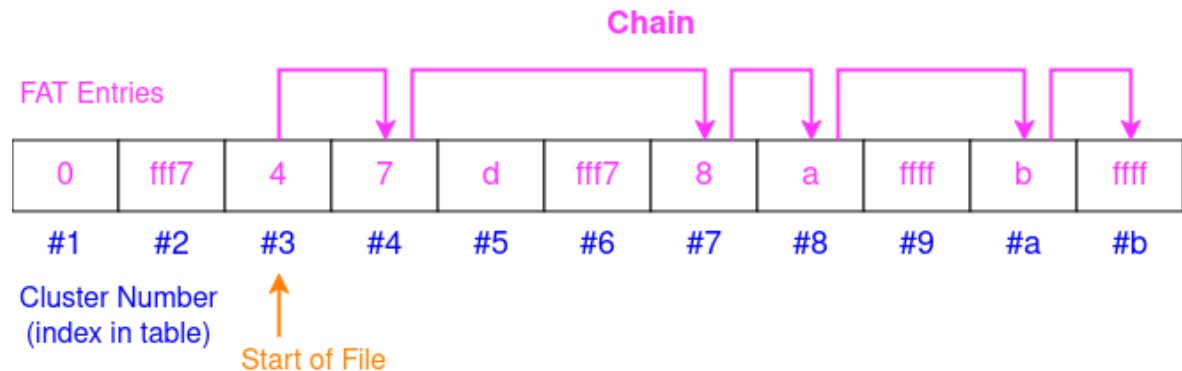
Example: FAT File System

- Fat was created in 1977; found widespread usage with the introduction of MS-DOS (1980)
- Cluster sizes: 512, 1024, 4096 Bytes, ..., 32 KiB
- Partition structure:
 - BSC = Bootsector → Optional Bootcode (not used anymore by UEFI)
 - FAT = File Allocation Table → Management of free and allocated blocks (fixed size, replicated for reliability)
 - Root Directory: Fixed size in FAT16, stored in BSC
 - File Cluster: Blocks containing file data



Example: FAT File System

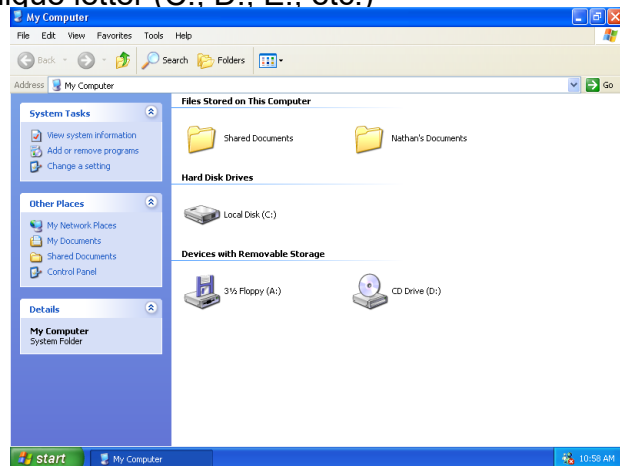
- Blocks are indexed via File Allocation Table (FAT)
 - Blocks are called *Clusters* in FAT
 - One entry for each cluster (16 Bit)
 - Copy of the FAT resides in main memory (for fast access)
- Clusters of a file are chained in the FAT
 - 0 = Empty cluster
 - 0xffff = Bad cluster
 - 0xffff = Last cluster of a file



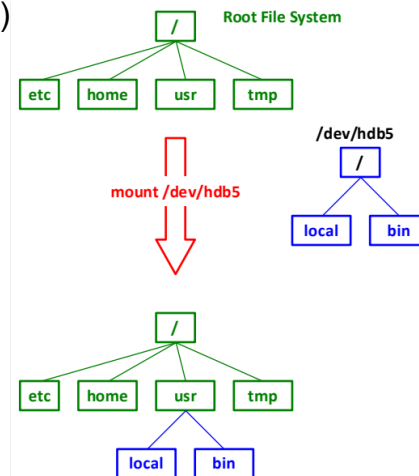
- Start of a file is stored in directory entry

- Multiple devices with physical filesystems can exist at the same time in a machine
- Devices can have different filesystems (e.g., *ext4* on a fixed SSD and *FAT32* on removable USB devices)
- Operating systems offer an abstraction over these physical filesystems
 - Uniform API for all file accesses, regardless of the physical filesystem

Windows: Disks are separate entities, identified by a unique letter (C:, D:, E:, etc.)



Unix: Disks can be mounted anywhere in the filesystem tree (transparent)



- Physical vs. Virtual Filesystems

- Filesystem API

- HeineOS Filesystem

- Summary

- C standard library defines unified API for filesystem accesses (`stdio.h`)
 - `fopen()`: Open a file and returns a `FILE` pointer for use in other file related functions
 - `fread()`: Read data from a file into a block of memory
 - `fwrite()`: Write data from a block of memory into a file
 - `ftell()`: Get the current read/write position of a `FILE` pointer
 - `fseek()`: Move the read/write position of a `FILE` pointer
- Operating system exposes system calls for file access (e.g., POSIX compatible)
 - APIs must be implemented separately for each operating system, using specific system calls

Example

```
const char *write_msg = "Hello, World!";
char read_msg[16];

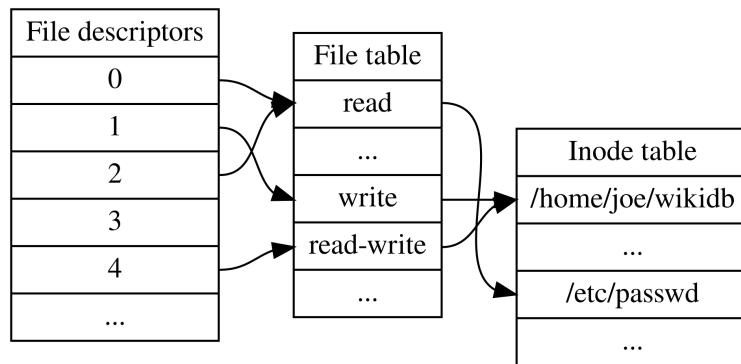
void main() {
    // Open file for reading and writing (create if not exists)
    FILE *file = fopen("hello.txt", "w+");
    if (file == NULL) {
        return;
    }

    fwrite(write_msg, 1, strlen(write_msg), file); // Write message to file
    fseek(file, 0, SEEK_SET); // Seek back to beginning of file

    // Read file content into buffer
    memset(read_msg, 0, sizeof(read_msg));
    fread(read_msg, 1, sizeof(read_msg), file);

    // Print read message and close file
    puts(read_msg);
    fclose(file);
}
```

- `open()` system call on Linux: `int open(const char *path, int flags)`
 - Return value is a *File Descriptor*, used to access the opened file in further system calls
- `read()` system call: `ssize_t read(int fd, void *buf, size_t count)`
 - `fd` must be a valid file descriptor, returned by a former call to `open()`
- All file related system call work on file descriptors
 - Each process has a table of open files; File descriptors are indices into this table



- Physical vs. Virtual Filesystems
- Filesystem API
- HeineOS Filesystem
- Summary

- Bootloader can load additional files (*modules*) along with the kernel into memory
- We use a TAR-archive as basis for our *read-only* filesystem
- Bootloader (towboot) loads the file into memory for us
- Access in `boot.rs` via `multiboot.find_tag::<ModuleTag>(TagType::Module)`

```
default = "heineos"                                loader/towboot.toml
timeout = 0
log_level = "info"

[entries]
  [entries.heineos]
    name = "HeineOS"
    image = "kernel.elf"
    modules = [ { image = "initrd.tar", argv = "initrd.tar" } ]
    argv = "log_level=debug log_to_terminal=true"
```

- TAR archives store multiple files sequentially in a single uncompressed archive
- Each file is preceded by a 512 byte header and its length padded to a multiple of 512
- We use the crate `tar-no-std` for parsing TAR archives

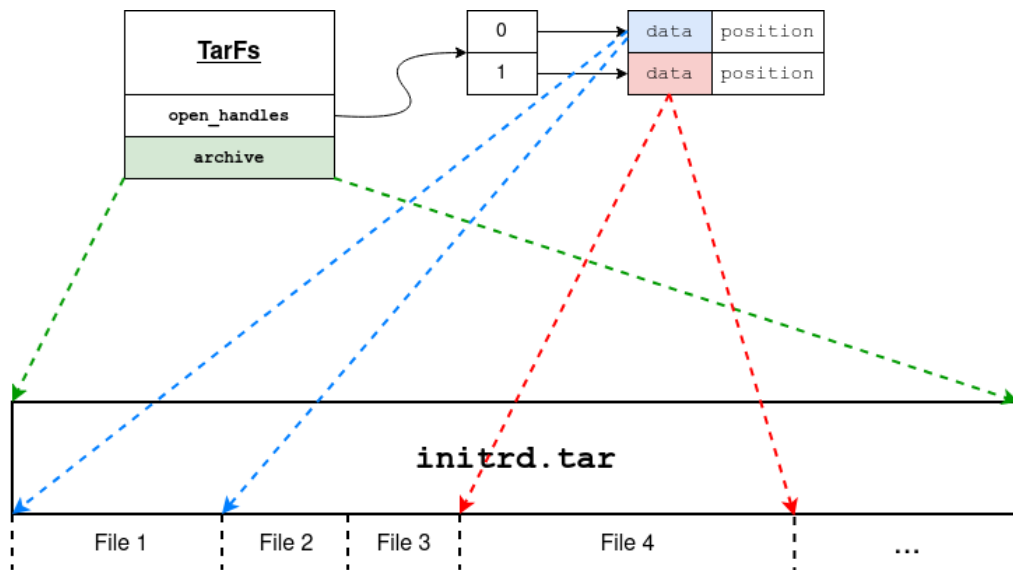
Field Offset	Field Size	Field
0	100	File path and name
100	8	File mode (octal)
108	8	Owner UID (octal)
116	8	Group GID (octal)
124	12	File size in bytes (octal)
136	12	Last mod. time (octal)
148	8	Header Checksum
156	1	Link indicator
157	100	Name of linked file

Original Unix TAR file header

■ API Example:

- `pub fn open(&self, mut path: &str) -> Result<FileHandle, FsError>`
- `pub fn read(&self, handle: FileHandle, buffer: &mut [u8]) -> Result<usize, FsError>`

- All files reside inside a TAR archive
- Open *file handles* (descriptors) are managed in `open_handles` table.
- `OpenFile` struct references data inside the TAR archive
- Conceptually similar to UNIX
→ `FileHandle` just wraps `usize` value
- Easily portable to user space via system calls (later!)



- Physical vs. Virtual Filesystems
- Filesystem API
- HeineOS Filesystem
- Summary

- Filesystems store files on (sector/block based) drives
- Allocation/Deallocation of sectors on a device
- Operating system manages open files (per process)
- User API uses system calls to access open files via file descriptors
- We do not have persistent storage in HeineOS and resort to a TAR archive
- File API is designed similar to UNIX-based operating systems